Every function should tell you what it does to the world

Gaze: a fixed-vocabulary effect system for AI-generated code

Roberta Carraro

The problem

AI agents write code that runs on your infrastructure.

You can review the PR. But can you see, at a glance, that the "search" tool also writes to the filesystem and shells out to `pip install`?

```
# CrewAI CodeInterpreterTool — real production code
def run_code_unsafe(self, code, libraries_used):
    for library in libraries_used:
        os.system(f"pip install {library}")   # shell injection
    exec(code, {}, exec_locals)               # arbitrary execution
```

The thesis

A fixed vocabulary of 10 effects is sufficient
to describe what any function does to the world.

Net touches the network	**Fs** reads or writes files	**Db** queries a database
Console terminal I/O	**Env** reads env vars	**Time** reads clock or sleeps
Rand randomness	**Async** concurrency	**Unsafe** subprocess, eval, FFI
Fail can exit		

Not extensible. Not configurable. Ten words. That's the opinion.

Why fixed?

Extensible effect systems exist. Koka, Unison, OCaml 5.
They're powerful. They're also hard to learn, hard to read,
and produce error messages that require a PhD to parse.

A fixed vocabulary is:

- **Learnable in minutes.** Ten words. You already know them.
- **Auditable by machines.** An LLM can read `can Net, Fs` and reason about it.
- **Proven at scale.** Deno's 8 permissions, Android's 12 permission groups, WASI's typed interfaces. Fixed vocabularies work for billions of devices.

The tradeoff: you can't say `can Log` or `can Cache`.
Domain concerns are types, not effects.

What we built

libgaze: a static effect analyzer for Python.

Two-pass analysis:

1. Walk the AST. Match calls against known functions and modules.
2. Propagate effects through the intra-module call graph.

```
$ libgaze check code_interpreter.py

_run:194  can Fs, Net, Unsafe
    calls run_code_unsafe (line 347)
    calls run_code_safety (line 281)
run_code_unsafe:347  can Unsafe
    os.system(f"pip install {library}")
    exec(code, {}, exec_locals)
```

Accuracy

Labeled benchmarks: 155 functions across both languages.

```
Python       101 functions   precision 100%   recall 100%
TypeScript    54 functions   precision 100%   recall 100%
```

Known boundary: method calls on injected objects (`self.driver.get(url)`, `container.exec_run()`) require type inference. Silence doesn't mean safe.

Scale

We built two analyzers (Python and TypeScript) and scanned 7 real agent framework repos.

3,211 files. 15,293 functions. Two languages.

	FILES	FUNCTIONS	PURE
Python (CrewAI, LangChain, AutoGPT)	1,604	12,511	66%
TypeScript (MCP Servers, Vercel AI, OpenAI Agents)	1,607	2,782	78%

The vocabulary didn't change between languages.
Not one effect was added, removed, or renamed.

What we found

71 functions combine `Unsafe` with other effects across the four repos.
Not just `exec()` alone, but `exec()` alongside `Net`, `Fs`, `Env`, `Console`.

Real findings from production agent tools:

- `os.system(f"pip install {library}")` — shell injection via f-string
- `exec(code, {}, exec_locals)` — arbitrary code execution
- `subprocess.run(["uv", "pip", "install", ...])` — auto-installs packages on import
- `__import__(name, ...)` — dynamic imports in a "sandbox"

Every one of these is in code that agents call autonomously.

The CI gate

```
# GitHub Action — gate PRs on effects
- uses: itchymutt/gaze/action@main
  with:
    path: src/tools/
    deny: Unsafe
```

The PR doesn't merge until the `Unsafe` effect is removed or the policy is updated to allow it. Non-zero exit code.

Works today. One line in your workflow file.

The language

libgaze reports effects. It tells you what your code does.

Gaze is a programming language that enforces them.
Same ten effects, different guarantee.
The library tells you what your code does.
The language makes it impossible to lie about it.

A function with no `can` clause is pure by construction.
If it tries to call `os.system()`, it doesn't compile.

The language is the proof that the vocabulary holds together
as a complete programming model. The library is the product.

Where this goes

Today: Two analyzers (Python + TypeScript), ~1,100 lines total. A GitHub Action for CI gating. A compiler that proves the vocabulary works as a type system.

Next: Publish to PyPI and npm. Cross-file analysis. Integration with runtime agent sandboxes to pre-check code against capability policies before launch.

The bet: the ten-effect vocabulary becomes a shared standard across languages, tools, and CI pipelines. Not because Gaze wins, but because the vocabulary is right.

Net, Fs, Db, Console, Env,

Time, Rand, Async, Unsafe, Fail

Ten words. Two languages. 15,293 functions.

The vocabulary held.

github.com/itchymutt/gaze